Webbportal med spring boot

Bakgrund

Wigellkoncernen har beslutat att samla några av affärstjänsterna under en portal för enklare administration och underhåll. Två grupper jobbar med projektet där en grupp ansvarar för användargränssnittet och en annan för de mikrotjänster som användargränssnittet kommunicerar med.

Ditt team arbetar med de mikrotjänster som ska utvecklas med Java Spring Boot

Lansering av portalen sker Måndag 13/4 kl 23:59 så samtliga projekt måste vara inlämnade då!

Arbetet består av två delar.

1. En individuell del.
   Du ansvarar själv över din egen mikrotjänst. Den ska fungera oberoende av de övrigas REST API:er och uppfylla betygskraven enligt nedan.
   Du redovisar din mikrotjänst i samband med gruppredovisningen.
   Du blir individuellt bedömd på din mikrotjänst
2. Ett grupparbete
   Inom gruppen ska ni tillsammans planera och genomföra de delarna som krävs för att ert mikrotjänstprojekt ska fungera som en helhet.
   Det betyder att er grupp ska utveckla en dashboard där ni kan övervaka ert system.
   Det betyder att er grupp ska utveckla en gateway som samtliga requests ska passera igenom.
   Det betyder också att om ni vill använda gemensam funktionalitet i någon form så kan den med fördel inkluderas i era api:er via en dependency, tänk shared-lib.
   Det betyder att ni redovisar portalen i grupp, och att även det momentet vägs in i det slutgiltiga betyget

Kravspecifikation Wigell Travels

Funktionella krav

API:et ska ha funktionalitet för både kunder och admin.

Tekniska krav

Skapa de klasser som behövs för att lösa uppgiften och är vettiga designmässigt från informationen i kraven.

Strukturera applikationen med lämpliga paket så som controller, service, entity osv.

Data ska lagras i en MySQL-databas. Databasen ska heta wigell_travel_db.

Verifiera med Postman att alla endpoints går att använda enligt specifikation.

För att visa totalt pris i SEK och PLN använder du en fast växelkurs (G), eller teamets egen mikrotjänst som hanterar konverteringen (VG)

Säkerhet

För G ska Basic security användas för autentisering och auktorisering.

För VG ska Keycloak användas för autentisering och auktorisering.

All admin funktionalitet ska kräva att användare är inloggad och har rollen ADMIN.

All kundfunktionalitet ska kräva inloggning med en användare som har rollen USER.

Skapa minst en användare med rollen USER.

Skapa minst en användare med rollen ADMIN.

Användarens id och lösenord hämtas från databasen. Lösenorden ska även vara krypterade med bcrypt (G)

Wigell Travels

Systemet ska kunna hantera kunder, resor, bokningar. Resorna ska ha pris per vecka, namn på hotell och destination (stad och land). Kunder ska ha unika ID-nummer, användarnamn och kunduppgifter som namn och adress. Bokningar ska inkludera avresedatum, destination, namn på hotell och totalpris. Priset ska presenteras i SEK och PLN.

Kunder

Kunderna ska kunna göra ett antal aktiviteter med följande endpoints:

- Lista resmål GET /api/v1/destinations
- Boka resa POST /api/v1/bookings
- Boka om resa PATCH /api/v1/bookings/{bookingId} (tillåtna fält: reslängd (veckor), destination, hotell)

• Se tidigare och aktiva bokningar GET /api/v1/bookings?customerId={customerId}

Admin

Administratörer ska kunna göra ett antal aktiviteter med följande endpoints:

• Lista kunder GET /api/v1/customers

• Lägga till kund POST /api/v1/customers

• Ta bort kund DELETE /api/v1/customers/{customerId}

• Uppdatera kund PUT /api/v1/customers/{customerId}

• Lägga till resmål POST /api/v1/destinations

• Ta bort resmål DELETE /api/v1/destinations/{destinationId}

• Uppdatera resmål PUT /api/v1/destinations/{destinationId}

• Lista resmål GET /api/v1/destinations

• Lägga till adress POST /api/v1/customers/{customerId}/addresses

• Ta bort adress DELETE /api/v1/customers/{customerId}/addresses/{addressId}

Viktigt att alla endpoints följer specifikationen eftersom de ska fungera med användargränssnittet.

Loggning

Applikationen ska logga info till konsolen (G) eller till fil (VG) när något skapas, ändras eller tas bort. Skriv ett meddelande som talar om vad som händer t.ex. ”admin created destination xxx”,

Övrigt

Ingen frontend ska implementeras. All kommunikation med API:et ska ske via Postman
API:et ska nås på port 8581

Java 24 och Spring Boot 3.x ska användas

Lombok är ej tillåtet att använda

Se till att det finns minst 5 kunder och 5 resmål i databasen vid redovisningens start

Datalagret använder Spring Data JPA; API följer REST-praxis (resurser, statuskoder, Location vid 201)

Actuators: health och info

Kravspecifikation Wigell Sushi

Funktionella krav

API:et ska ha funktionalitet för både kunder och admin

Tekniska krav

Skapa de klasser som behövs för att lösa uppgiften och är vettiga designmässigt från informationen i kraven.

Strukturera applikationen med lämpliga paket så som controller, service, entity osv.

Data ska lagras i en MySQL-databas. Databasen ska heta wigell_sushi_db.

Verifiera med Postman att alla endpoints går att använda enligt specifikation.

För att visa totalt pris i SEK och JPY använder du en fast växelkurs (G), eller teamets egen mikrotjänst som hanterar konverteringen (VG)

Säkerhet

För G ska Basic security användas för autentisering och auktorisering.

För VG ska Keycloak användas för autentisering och auktorisering.

All admin funktionalitet ska kräva att användare är inloggad och har rollen ADMIN.

All kundfunktionalitet ska kräva inloggning med en användare som har rollen USER.

Skapa minst en användare med rollen USER.

Skapa minst en användare med rollen ADMIN.

Användarens id och lösenord hämtas från databasen. Lösenorden ska även vara krypterade med bcrypt (G)

Wigell Sushi

Systemet ska kunna hantera kunder, bokningar, beställningar. Beställningar ska ha totalpris, kunduppgifter och rätter. Beställning avser takeaway. Kunder ska ha unika ID-nummer, användarnamn och kunduppgifter som namn och adress. Bokningar ska inkludera antal gäster, lokal, önskad förtäring (maträtter och antal) och totalpris. Priset ska presenteras i SEK och JPY. Lokaler ska innehålla information om max antal gäster samt ev teknisk utrustning

Kunder

Kunderna ska kunna göra ett antal aktiviteter med följande endpoints:

- Lista rätter GET /api/v1/dishes

- Beställ takeaway POST /api/v1/orders
- Reservera lokal POST /api/v1/bookings
- Uppdatera bokning PATCH /api/v1/bookings/{bookingId} (tillåtna fält: datum, antal rätter, lokal, önskad förtäring, teknisk utrustning)
- Se tidigare och aktiva bokningar GET /api/v1/bookings?customerId={customerId}
- Hämta beställningar GET /api/v1/orders?customerId={customerId}

Admin

Administratörer ska kunna göra ett antal aktiviteter med följande endpoints:

- Lista kunder GET /api/v1/customers
- Lägga till kund POST /api/v1/customers
- Ta bort kund DELETE /api/v1/customers/{customerId}
- Uppdatera kund PUT /api/v1/customers/{customerId}
- Hämta maträtt GET /api/v1/dishes/{dishId}
- Uppdatera maträtt PUT /api/v1/dishes/{dishId}
- Lägga till maträtt POST /api/v1/dishes
- Ta bort maträtt DELETE /api/v1/dishes/{dishId}
- Lista lokaler GET /api/v1/rooms
- Hämta lokal GET /api/v1/rooms/{roomId}
- Skapa lokal POST /api/v1/rooms
- Uppdatera lokal PUT /api/v1/rooms/{roomId}
- Ta bort lokal DELETE /api/v1/rooms/{roomId}
- Lista rätter GET /api/v1/dishes
- Hämta order GET /api/v1/orders/{orderId}
- Lägga till adress POST /api/v1/customers/{customerId}/addresses
- Ta bort adress DELETE /api/v1/customers/{customerId}/addresses/{addressId}

Viktigt att alla endpoints följer specifikationen eftersom de ska fungera med användargränssnittet.

Loggning

Applikationen ska logga info till konsolen (G) eller till fil (VG) när något skapas, ändras eller tas bort. Skriv ett meddelande som talar om vad som händer t.ex. ”admin deleted dish xxx”

Övrigt

Ingen frontend ska implementeras. All kommunikation med API:et ska ske via Postman

API:et ska nås på port 8582

Java 24 och Spring Boot 3.x ska användas

Lombok är ej tillåtet att använda

Se till att det finns minst 5 kunder, 5 rätter, två beställningar och 3 lokaler i databasen vid redovisningens start

Datalagret använder Spring Data JPA; API följer REST-praxis (resurser, statuskoder, Location vid 201)

Actuators: health och info

Kravspecifikation Wigell Cinema

Funktionella krav

API:et ska ha funktionalitet för både kunder och admin

Tekniska krav

Skapa de klasser som behövs för att lösa uppgiften och är vettiga designmässigt från informationen i kraven.

Strukturera applikationen med lämpliga paket så som controller, service, entity osv.

Data ska lagras i en MySQL-databas. Databasen ska heta wigell_cinema_db.

Verifiera med Postman att alla endpoints går att använda enligt specifikation.

För att visa totalt pris i SEK och USD använder du en fast växelkurs (G), eller teamets egen mikrotjänst som hanterar konverteringen (VG)

Säkerhet

För G ska Basic security användas för autentisering och auktorisering.

För VG ska Keycloak användas för autentisering och auktorisering.

All admin funktionalitet ska kräva att användare är inloggad och har rollen ADMIN.

All kundfunktionalitet ska kräva inloggning med en användare som har rollen USER.

Skapa minst en användare med rollen USER.

Skapa minst en användare med rollen ADMIN.

Användarens id och lösenord hämtas från databasen. Lösenorden ska även vara krypterade med bcrypt (G)

Wigell Cinema

Systemet ska kunna hantera kunder, bokningar, biljettförsäljning. Kunder ska ha unika ID-nummer, användarnamn och kunduppgifter som namn och adress. Bokningar ska inkludera antal gäster, lokal, önskad föreställning (film eller egen talare) och totalpris. Priset ska presenteras i SEK och USD. Bokning avser en hel salong. Lokaler ska innehålla information om max antal gäster samt ev teknisk utrustning Filmer ska innehålla information om åldersgräns, längd samt titel och genre

Kunder

Kunderna ska kunna göra ett antal aktiviteter med följande endpoints:

• Lista filmer GET /api/v1/movies

• Reservera lokal POST /api/v1/bookings

- Köpa biljett POST /api/v1/tickets
- Se biljetter GET /api/v1/tickets?customerId={customerId}
- Lista föreställningar GET /api/v1/screenings?movieId={movieId}&date={YYYY-MM-DD}
- Uppdatera bokning PATCH /api/v1/bookings/{bookingId} (tillåtna fält: datum, teknisk utrustning)
- Se tidigare och aktiva bokningar GET /api/v1/bookings?customerId={customerId}

Admin

Administratörer ska kunna göra ett antal aktiviteter med följande endpoints:

- Lista kunder GET /api/v1/customers
- Lägga till kund POST /api/v1/customers
- Ta bort kund DELETE /api/v1/customers/{customerId}
- Uppdatera kund PUT /api/v1/customers/{customerId}
- Lägga till film POST /api/v1/movies
- Ta bort film DELETE /api/v1/movies/{movieId}
- Lista lokaler GET /api/v1/rooms
- Hämta lokal GET /api/v1/rooms/{roomId}
- Lägg till lokal POST /api/v1/rooms
- Uppdatera lokal PUT /api/v1/rooms/{roomId}
- Lista filmer GET /api/v1/movies
- Hämta film GET /api/v1/movies/{movieId}
- Lista föreställningar GET /api/v1/screenings
- Lägg till föreställning POST /api/v1/screenings
- Ta bort föreställning DELETE /api/v1/screenings/{screeningId}
- Lägga till adress POST /api/v1/customers/{customerId}/addresses
- Ta bort adress DELETE /api/v1/customers/{customerId}/addresses/{addressId}

Viktigt att alla endpoints följer specifikationen eftersom de ska fungera med användargränssnittet.

Loggning

Applikationen ska logga info till konsolen (G) eller till fil (VG) när något skapas, ändras eller tas bort. Skriv ett meddelande som talar om vad som händer t.ex. ”admin deleted movie xxx”

Övrigt

Ingen frontend ska implementeras. All kommunikation med API:et ska ske via Postman

API:et ska nås på port 8583

Java 24 och Spring Boot 3.x ska användas

Lombok är ej tillåtet att använda

Se till att det finns minst 5 kunder, 5 filmer, två bokningar och 3 lokaler i databasen vid redovisningens start

Datalagret använder Spring Data JPA; API följer REST-praxis (resurser, statuskoder, Location vid 201)

Actuators: health och info

Kravspecifikation Wigell Padel

Funktionella krav

API:et ska ha funktionalitet för både kunder och admin

Tekniska krav

Skapa de klasser som behövs för att lösa uppgiften och är vettiga designmässigt från informationen i kraven.

Strukturera applikationen med lämpliga paket så som controller, service, entity osv.

Data ska lagras i en MySQL-databas. Databasen ska heta wigell_padel_db.

Verifiera med Postman att alla endpoints går att använda enligt specifikation.

För att visa totalt pris i SEK och EUR använder du en fast växelkurs (G), eller teamets egen mikrotjänst som hanterar konverteringen (VG)

Säkerhet

För G ska Basic security användas för autentisering och auktorisering.

För VG ska Keycloak användas för autentisering och auktorisering.

All admin funktionalitet ska kräva att användare är inloggad och har rollen ADMIN.

All kundfunktionalitet ska kräva inloggning med en användare som har rollen USER.

Skapa minst en användare med rollen USER.

Skapa minst en användare med rollen ADMIN.

Användarens id och lösenord hämtas från databasen. Lösenorden ska även vara krypterade med bcrypt (G)

Wigell Padel

Systemet ska kunna hantera kunder, bokningar, se tillgänglighet och information om banorna. Kunder ska ha unika ID-nummer, användarnamn och kunduppgifter som namn och adress. Bokningar ska inkludera antal spelare, bana, datum, tid och totalpris. Priset ska presenteras i SEK och EUR.

Kunder

Kunderna ska kunna göra ett antal aktiviteter med följande endpoints:

- Lista lediga tider GET /api/v1/availability?date={YYYY-MM-DD}&courtId={courtId}
- Boka tid POST /api/v1/bookings

- Uppdatera bokning PATCH /api/v1/bookings/{bookingId} (tillåtna fält: datum, tid, antal spelare, bana)
- Lista bokningar GET /api/v1/bookings?customerId={customerId}

Admin

Administratörer ska kunna göra ett antal aktiviteter med följande endpoints:

- Lista kunder GET /api/v1/customers
- Lägga till kund POST /api/v1/customers
- Ta bort kund DELETE /api/v1/customers/{customerId}
- Uppdatera kund PUT /api/v1/customers/{customerId}
- Hämta padelbanor GET /api/v1/courts
- Hämta padelbana GET /api/v1/courts/{courtId}
- Lägga till padelbana POST /api/v1/courts
- Uppdatera information om banan PUT /api/v1/courts/{courtId}
- Ta bort padelbana DELETE /api/v1/courts/{courtId}
- Ta bort en bokning DELETE /api/v1/bookings/{bookingId}
- Lista samtliga bokningar GET /api/v1/bookings
- Hämta enskild bokning GET /api/v1/bookings/{bookingId}
- Lägga till adress POST /api/v1/customers/{customerId}/addresses
- Ta bort adress DELETE /api/v1/customers/{customerId}/addresses/{addressId}

Viktigt att alla endpoints följer specifikationen eftersom de ska fungera med användargränssnittet.

Loggning

Applikationen ska logga info till konsolen (G) eller till fil (VG) när något skapas, ändras eller tas bort. Skriv ett meddelande som talar om vad som händer t.ex. ”admin deleted booking xxx”

Övrigt

Ingen frontend ska implementeras. All kommunikation med API:et ska ske via Postman

API:et ska nås på port 8584

Java 24 och Spring Boot 3.x ska användas

Lombok är ej tillåtet att använda

Se till att det finns minst 5 kunder, 5 banor och två bokningar i databasen vid redovisningens start

Datalagret använder Spring Data JPA; API följer REST-praxis (resurser, statuskoder, Location vid 201)

Actuators: health och info

Kravspecifikation Wigell MC Rental

Funktionella krav

API:et ska ha funktionalitet för både kunder och admin

Tekniska krav

Skapa de klasser som behövs för att lösa uppgiften och är vettiga designmässigt från informationen i kraven.

Strukturera applikationen med lämpliga paket så som controller, service, entity osv.

Data ska lagras i en MySQL-databas. Databasen ska heta wigell_mc_db.

Verifiera med Postman att alla endpoints går att använda enligt specifikation.

För att visa totalt pris i SEK och GBP använder du en fast växelkurs (G), eller teamets egen mikrotjänst som hanterar konverteringen (VG)

Säkerhet

För G ska Basic security användas för autentisering och auktorisering.

För VG ska Keycloak användas för autentisering och auktorisering.

All admin funktionalitet ska kräva att användare är inloggad och har rollen ADMIN.

All kundfunktionalitet ska kräva inloggning med en användare som har rollen USER.

Skapa minst en användare med rollen USER.

Skapa minst en användare med rollen ADMIN.

Användarens id och lösenord hämtas från databasen. Lösenorden ska även vara krypterade med bcrypt (G)

Wigell MC Rental

Systemet ska kunna hantera kunder, bokningar, se tillgänglighet och information om motorcyklarna. Kunder ska ha unika id-nummer, användarnamn och kunduppgifter som namn och adress. Bokningar ska inkludera kunduppgifter, motorcykel, datum, tid och totalpris. Priset ska presenteras i SEK och GBP.

Kunder

Kunderna ska kunna göra ett antal aktiviteter med följande endpoints:

- Lista lediga motorcyklar GET /api/v1/availability?from={YYYY-MM-DD}&to={YYYY-MM-DD}
- Hyr motorcykel POST /api/v1/bookings
- Uppdatera bokning PATCH /api/v1/bookings/{bookingId} (tillåtna fält: motorcykel, datum)

• Lista bokningar GET /api/v1/bookings?customerId={customerId}

Admin

Administratörer ska kunna göra ett antal aktiviteter med följande endpoints:

• Lista kunder GET /api/v1/customers

• Hämta kund GET /api/v1/customers/{customerId}

• Lägga till kund POST /api/v1/customers

• Ta bort kund DELETE /api/v1/customers/{customerId}

• Uppdatera kund PUT /api/v1/customers/{customerId}

• Lägga till motorcykel POST /api/v1/bikes

• Ta bort motorcykel DELETE /api/v1/bikes/{bikeId}

• Uppdatera motorcykel PUT /api/v1/bikes/{bikeId}

• Lista samtliga motorcyklar GET /api/v1/bikes

• Hämta en motorcykel GET /api/v1/bikes/{bikeId}

• Lista bokningar GET /api/v1/bookings

• Hämta bokning GET /api/v1/bookings/{bookingId}

• Uppdatera bokning PUT /api/v1/bookings/{bookingId}

• Ta bort en bokning DELETE /api/v1/bookings/{bookingId}

• Returnera bokning PATCH /api/v1/bookings/{bookingId} (tillåtna fält: status)

• Lägga till adress POST /api/v1/customers/{customerId}/addresses

• Ta bort adress DELETE /api/v1/customers/{customerId}/addresses/{addressId}

Viktigt att alla endpoints följer specifikationen eftersom de ska fungera med användargränssnittet.

Loggning

Applikationen ska logga info till konsolen (G) eller till fil (VG) när något skapas, ändras eller tas bort. Skriv ett meddelande som talar om vad som händer t.ex. ”admin deleted bike xxx”

Övrigt

Ingen frontend ska implementeras. All kommunikation med API:et ska ske via Postman

API:et ska nås på port 8585

Java 24 och Spring Boot 3.x ska användas

Lombok är ej tillåtet att använda

Se till att det finns minst 5 kunder, 5 motorcyklar och två bokningar i databasen vid redovisningens start

Datalagret använder Spring Data JPA; API följer REST-praxis (resurser, statuskoder, Location vid 201)

Actuators: health och info